

Become a Guru in React.js

Master Modern Frontend Development with React.js

Author: Luv, Andrey

Table of Contents

1. Introduction to React.js
2. Setting Up the React Environment
3. Understanding Components
4. JSX and Rendering
5. Props and State
6. Event Handling
7. React Hooks
8. Working with Forms
9. React Router
10. API Integration
11. State Management
12. Performance Optimization
13. React Best Practices
14. Building Scalable Applications
15. Testing React Applications
16. Real-World React Projects
17. Conclusion

Introduction

React.js is one of the most popular JavaScript libraries for building modern user interfaces and single-page applications. Developed by Meta (Facebook), React allows developers to create fast, reusable, and scalable frontend applications.

Today, React is used by companies worldwide to power applications such as dashboards, e-commerce platforms, social media systems, and enterprise software.

This ebook is designed to help developers move beyond the basics and master advanced React.js concepts used in professional frontend development.

Chapter 1 — Introduction to React.js

React is a JavaScript library for building user interfaces.

Why Developers Love React

React offers:

- Component-based architecture
- Fast rendering with Virtual DOM
- Reusable UI components
- Strong ecosystem
- Excellent developer experience

Applications Built with React

- Social media apps
- E-commerce websites
- Admin dashboards
- Chat applications
- Streaming platforms

Chapter 2 — Setting Up the React Environment

Required Tools

- Node.js
- npm
- VS Code or IntelliJ IDEA

Install Node.js

Download Node.js from:

[Node.js Official Website](https://nodejs.org/)

Creating a React Project with Vite

```
npm create vite@latest
```

Installing Dependencies

```
npm install  
npm run dev
```

Vite provides a fast development environment for React applications.

Chapter 3 — Understanding Components

Components are reusable building blocks in React.

Functional Component Example

```
function Welcome() {  
  return <h1>Welcome to React</h1>;  
}  
  
export default Welcome;
```

Using the Component

```
import Welcome from './Welcome';
```

```
function App() {  
  return <Welcome />;  
}
```

Chapter 4 — JSX and Rendering

JSX allows developers to write HTML-like syntax inside JavaScript.

JSX Example

```
const element = <h1>Hello React</h1>;
```

Rendering Lists

```
const books = ["JavaScript", "React", "Spring Boot"];
```

```
function App() {  
  return (  
    <ul>  
      {books.map(book => (  
        <li key={book}>{book}</li>  
      ))}  
    </ul>  
  );  
}
```

Chapter 5 — Props and State

Props

Props allow data transfer between components.

```
function User(props) {  
  return <h1>{props.name}</h1>;  
}
```

State

State stores dynamic data inside components.

```
import { useState } from "react";
```

```
function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      {count}
    </button>
  );
}
```

Chapter 6 — Event Handling

React handles events similarly to JavaScript.

Button Click Event

```
function App() {

  const handleClick = () => {
    alert("Button clicked");
  };

  return (
    <button onClick={handleClick}>
      Click
    </button>
  );
}
```

Chapter 7 — React Hooks

Hooks simplify state and lifecycle management.

useEffect Example

```
import { useEffect } from "react";

useEffect(() => {
  console.log("Component mounted");
}, []);
```

useMemo Example

```
const expensiveCalculation = useMemo(() => {  
  return calculateTotal();  
}, []);
```

useRef Example

```
const inputRef = useRef();
```

Chapter 8 — Working with Forms

Controlled Input Example

```
import { useState } from "react";  
  
function Form() {  
  const [email, setEmail] = useState("");  
  
  return (  
    <input  
      value={email}  
      onChange={(e) => setEmail(e.target.value)}  
    />  
  );  
}
```

Chapter 9 — React Router

React Router enables page navigation.

Installation

```
npm install react-router-dom
```

Basic Routes

```
import {  
  BrowserRouter,  
  Routes,  
  Route  
} from "react-router-dom";
```

Example Route

```
<Routes>
  <Route path="/" element={<Home />} />
  <Route path="/about" element={<About />} />
</Routes>
```

Chapter 10 — API Integration

React applications often consume backend APIs.

Fetch Example

```
useEffect(() => {

  fetch("https://jsonplaceholder.typicode.com/posts")
    .then(response => response.json())
    .then(data => console.log(data));

}, []);
```

Axios Example

```
npm install axios
```

```
import axios from "axios";

axios.get("/api/books")
  .then(response => console.log(response.data));
```

Chapter 11 — State Management

Large applications require centralized state management.

Context API

```
const UserContext = createContext();
```

Redux Toolkit

Install Redux Toolkit:

```
npm install @reduxjs/toolkit react-redux
```

Redux helps manage global application state efficiently.

Chapter 12 — Performance Optimization

Performance is critical in modern applications.

React.memo

```
export default React.memo(Component);
```

Lazy Loading

```
const Dashboard = React.lazy(() =>
  import("../Dashboard")
);
```

Code Splitting

Code splitting improves application loading speed.

Chapter 13 — React Best Practices

Keep Components Small

Smaller components are easier to maintain.

Organize Project Structure

```
src/
├── components/
├── pages/
├── services/
├── hooks/
├── context/
└── assets/
```

Use Reusable Components

Avoid duplicating UI logic.

Chapter 14 — Building Scalable Applications

Professional React applications require scalable architecture.

Recommended Architecture

- Component-based design
- Reusable hooks
- API service layers
- Modular folder structure

Clean Code Principles

- Use meaningful variable names
- Avoid deeply nested logic
- Separate business logic from UI

Chapter 15 — Testing React Applications

Testing improves reliability.

React Testing Library

Install:

```
npm install @testing-library/react
```

Simple Test Example

```
test("renders title", () => {  
  render(<App />);  
});
```

Chapter 16 — Real-World React Projects

Beginner Projects

1. Todo List
2. Calculator App
3. Weather Application
4. Notes App

Intermediate Projects

- E-library frontend
- E-commerce application
- Chat application
- Movie streaming frontend

Advanced Projects

- Admin dashboard
- Real-time collaboration platform
- SaaS management system

Chapter 17 — Conclusion

React.js has become one of the most important technologies in frontend development. Mastering React allows developers to create scalable, maintainable, and high-performance applications used in real-world environments.

Becoming a React Guru requires:

- consistent practice,
- project building,
- understanding modern React patterns,
- learning performance optimization,
- mastering state management.

The more applications you build, the stronger your React skills become.

Final Advice

- Build projects continuously
- Learn modern React Hooks deeply
- Practice API integration
- Understand component architecture
- Explore TypeScript with React
- Study advanced state management

End

Become a Guru in React.js — Professional Frontend Development Guide